HumanTrace2Workflow&Verifier

从一个人已有的访问记录里编译出他的 workflow 与 verifier

Zhi Zheng

新加坡国立大学

工作草稿。结果是初步的：每种配置只跑了一次，样本为合成数据。

摘要

人在电脑上重复着大量流程，但要把其中一个交给 agent，仍然得自己描述、自己写 prompt、自己检查结果。最近的产品能把一次录下来的演示变成可复用的 skill，最近的研究能从被动记录的轨迹里归纳出任务模型。两者恢复的都是一件事怎么做，都没有恢复这个人认为怎样才算做对。我们认为这两者不可分：没有最优的 workflow，只有在某个人的验收标准之下最优的 workflow，所以要学的对象是一对 (workflow, verifier)。verifier 的证据不在任务的主线上，而在主线之外：纠正、中断、复核、返工；其中很大一部分已经写在这个人所用的 coding agent 的日志里。HT2WV 在逐项授权之下采集这类访问记录，确定性地把它们压缩成一条时间线和一批“主线之外”的信号候选，然后在已有的 coding agent 里开无头会话，蒸馏出一份画像，并为每个任务蒸馏出一个以多会话 prompt 形式交付的 workflow，以及一个分层的 verifier——每个会话拿到的都是同一份由它渲染成的 prompt。为了度量，我们构造了合成样本：先预填一份个人风格，据此写出记录，再把风格藏起来；系统按两件事打分：它的 workflow 满足了这个人的 rubric 中的几条，它的 verifier 抓住了几条被隐藏的风格。其中有一个持续七天、带状态的样本，在里面把对的事按错的顺序做，结果就会不同。初步结果是：单步任务第一次执行就饱和了，七天的样本七天里填满了三天；而在我们跑过的五个样本里，有三个学出来的 verifier 会拒绝这个人本来会留下的产出——只有覆盖率、却不接受参考产出的，不能算 verifier。三条边里只有“从人蒸馏 verifier”这一步会读到个人的原始记录；因此我们把这种样本格式看作训练本地开源模型的材料，把许多不同的人的 (workflow, verifier) 看作一个多样化、能自我改进的 working group 的成员——那是本文之后的下一个阶段。

1 引言

Computer-use agent 已经能操作界面，但它该做什么、结果要好到什么程度，仍然由人手工指定。有两条路线在减轻前一个负担：当前 agent 产品里的“录制并回放”功能把一次带讲解的演示变成 skill [1, 2, 3]；任务模型归纳则从被动记录的轨迹里恢复目标层级和过程 [4]。两者产出的都是对流程的描述。后一个负担——判断结果能不能接受——仍然留给了人。

而这个标准是因人而异的。同样是写周报，两个人在“怎样才算合格”上可以完全不同：一个人在每个数字都对着源文件核过、并在末尾写明数据来源之前，什么都不会发；另一个人要的是一段语气亲切的结论。为其中一个人优化出来的 agent，对另一个人就是错的（图 2a）。我们把这一点当真：

> *workflow 与 verification 是耦合的。要学的不是一个流程，而是一对 (workflow, verifier)，而且 verifier 属于某一个人。*

为什么要做这个系统：是三角，不是流水线。 这里有三样东西，每一样作用于下一样（图 1）。人指导 *verifier*：他纠正了什么、拒绝了什么、回头核对了什么、在 rubric 上勾了什么，标准就是从这里来的；没有他的批准，标准的任何改动都不生效。*verifier* 优化并加速 *workflow*：它是搜索 workflow 时的目标函数；同时它作为同一份 prompt 附给每一个 agent 会话，使一个很长的流程可以拆到多个会话里去做，而各个会话对“做对了”的理解不会走散。*workflow* 给人提效：它接手重复性的工作，并按这个人读得最快的方式汇报回来。

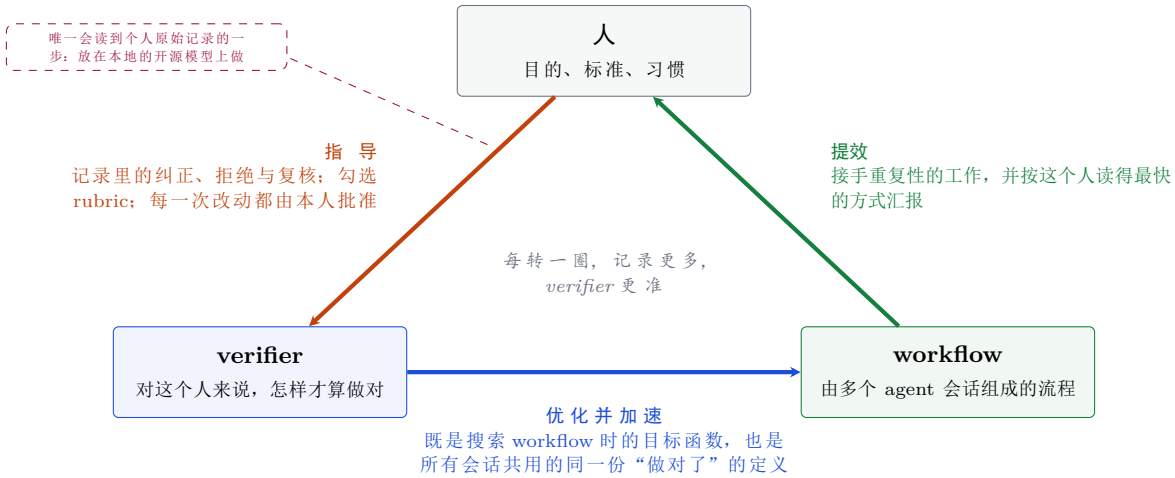


图 1: 为什么要有这个系统。人指导 verifier，verifier 优化并加速 workflow，workflow 把时间还给人。三条边里只有第一条会接触到个人的原始记录。

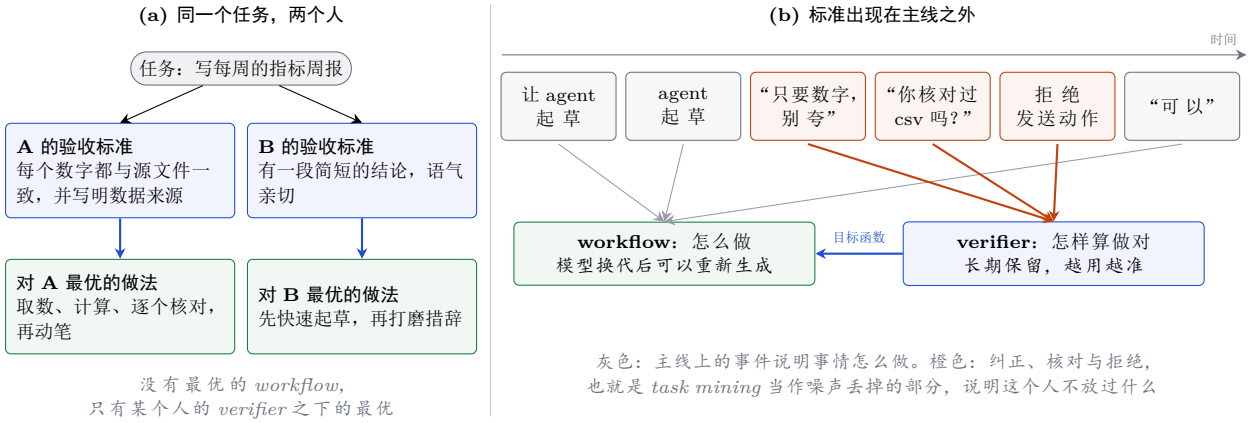


图 2: 立意。(a) 验收标准因人而异，而它决定了哪个 workflow 最优。(b) 这个标准的证据在任务主线之外，在这个人与 coding agent 已有的对话里最密。

没有 verifier，人就只能直接去指导 workflow，而且每次运行都要指导一遍——那就是写 prompt，它不会累积。有了 verifier，人的精力只在标准上花一次，此后这个循环每转一圈，都会留下更多记录，用来把标准磨得更准。

由此得到三个推论。个人化不是附加功能，而是问题定义本身。verifier 才是持久的资产：模型变强之后 workflow 可以重新生成，甚至换成完全不同的做法，而一个人接受什么变化得很慢。优化也因此有了明确的定义：固定这个人的 verifier，在 workflow 的空间里搜索。

verifier 的证据在哪里。 不在任务的主线上，而在 task mining 当作噪声丢掉的部分：撤销重做、看一眼来源却什么都没复制、发送之前的迟疑；最密集的，是这个人怎么跟他的 coding agent 说话。每一句“这不对”“你核对过吗”，每一次中断、每一次拒绝工具调用，都是他用自己的话说出来的验收标准，带时间戳，被打回的产出就在上一条消息里（图 2b）。这些日志本来就在机器上，不需要额外录制任何东西。

本文所处的位置。 我们把“用 coding 模型把工作自动化”看成五个阶段（图 3）：*coding*，人在模型的帮助下写；*coding agent*，一个 agent 把一件任务从头做到尾；*coding multi-agent workflow*，多个 agent 会话

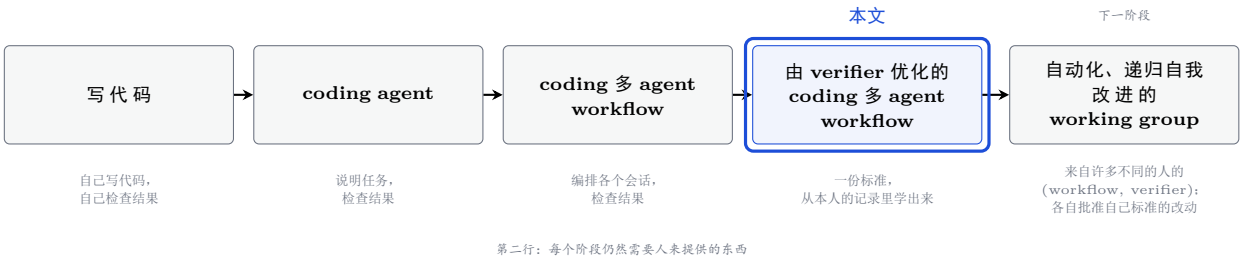


图 3: 我们设想的发展脉络。每个阶段都拿走一样原本要由人提供的东西。本文是第四阶段：多会话的 workflow 在一个从本人记录里学出来的 verifier 之下被优化。第五阶段把从许多不同的人那里蒸馏出来的 (workflow, verifier) 组成一个小组。

被编排成一个流程；*verifier-optimized coding multi-agent workflow*，这种编排不再由人手工完成，而是在一个明确的验收标准之下搜索出来；以及 *automated (recursive self-improving) working group*，一个常设的小组，长期维护并改进它自己的 workflow。每一个阶段都拿走一样原本要由人提供的东西：敲键盘、一步步盯着、编排步骤，最后是对编排本身的维护。本文实现的是第四个阶段。让第四阶段成为可能的那个对象，也正是让第五阶段可以放心去尝试的那个对象：一个明确的、可执行的、归本人所有的 verifier。有了它，被改进的东西（workflow）和评判它的东西（verifier）就被分开了；一个会改写自己流程的小组，仍然要对一份它自己无权单方面改写的标准负责。

下一个阶段的关键是多样性。 一个人的一对 (workflow, verifier)，只能造出一个照着这个人的样子工作的成员。我们想要的 working group，是由从许多不同的人那里蒸馏出来的许多对组成的：那个数字没核对就不发出去的分析师，那个没有回归测试就不合并修复的工程师，那个按“谁先申请”来排队的售后运营。他们的 workflow 给小组带来一系列不同的做法；他们的 verifier 给小组带来一系列不同的标准，这些标准之间的分歧是有用的——在一个人的标准下做出来的东西，可以放到另一个人的标准下去审。这样的小组里的多样性不需要靠写 persona 的 prompt 凭空造出来：它是从这些 (workflow, verifier) 背后的人那里继承来的，而且始终可以度量，因为每个成员的标准都是一个明确的、可执行的对象。

什么必须是私密的。 图 1 的三条边里，只有一条会读到个人的原始记录：从人蒸馏出 verifier。执行 workflow 看到的只是手头这件任务；优化 workflow 看到的是 verifier，而它是一段很短的、可以审阅的文本。这两步都不需要这个人的历史。因此隐私要求只落在一个边界清楚的步骤上，而这一步就应该放在本人自己托管的模型上做。下文介绍的那种样本——记录，配上生成这些记录的风格——正是这样的模型所需要的监督信号；所以我们正在收集的数据集，同时也是训练能在本地运行的开源模型（Qwen 27B、35B-A3B 这个量级）的材料。更强的托管模型则可以用在执行和优化上，那里不暴露任何私密的东西。

贡献。

- 把个人自动化表述为“人—verifier—workflow”的三角关系，其中要学的对象是一对 (workflow, verifier)；verifier 分五层（客观、顺序、风格、偏好、汇报），外加一份“这个人怎么查”的检查方式（§4）。
- HT2WV：一个开源实现。它在逐项授权之下读取一个人已有的访问记录；自己不调用任何模型 API，所有的阅读和写作都交给已有 coding agent 的无头会话去做，再用确定性的检查卡住这些会话的产出。
- 一种带有隐藏的、预填的个人风格的样本格式与打分方式，其中包括一个对顺序敏感、持续七天、带状态的任务；以及在六个样本中的五个上的初步测量（§5–6）。

2 问题

一个人 u 留下访问记录 R_u：与 coding agent 的对话、shell 历史、浏览器历史、导出的聊天记录、文件名与修改时间，以及可选的屏幕录制。对一件反复出现的任务，这个人有一个目的，并且说得出一份简短的 *rubric*

Q_u——他会逐条去勾的验收项。在它背后是一份更大的风格 S_u：他在用、却很少说出口的标准，有的关于结果，有的关于做事的先后顺序，有的关于别人该怎么向他汇报。

系统从 R_u 和 Q_u 产出：

- 一个 *workflow* W：一个有序的 prompt 列表，每个 prompt 对应一个 agent 会话；会话之间不共享记忆，只通过文件交接；
- 一个 *verifier* V：一组条目，每条是一个判断，配有可执行的检查脚本或评审用的 rubric，带有所属的层、权重，以及指向支撑它的记录的指针；外加一份检查方式（policy），描述这个人怎么查：查多深、什么时候查、失败了怎么办、什么情况下停下来问人。

V 被渲染成一份 prompt，W 的每一个会话都会拿到它。于是流程可以切成任意多个会话，但“做对了”只有一份定义。

在一个留出的任务实例上（其参考产出为 $y^\star$，即 u 自己会留下的那一份），我们度量三件事：

$$\begin{aligned}
\textbf{rubric 完成度}: &\quad |\{q \in Q_u : q(W(x)) \text{ 成立}\}| \,/\, |Q_u|, \\
\textbf{风格覆盖率}: &\quad |\{s \in S_u : V \text{ 在某个对} y^\star \text{ 判通过的条目上判} y^\star_{\neg s} \text{ 不通过}\}| \,/\, |S_u|, \\
\textbf{参考产出被接受}: &\quad V(y^\star) = \text{accept},
\end{aligned}$$

其中 $y^\star_{\neg s}$ 是把参考产出恰好按 s 所禁止的方式改坏之后的版本。覆盖率奖励严格的 verifier；“参考产出被接受”则惩罚那些在错误的地方严格的 verifier。对长程任务，rubric 完成度按天报告，主要的数字是有几天被填满，即当天适用的 rubric 条目全部通过。

3 一个例子：一周的退款

图 4 是我们最难的一个样本。许晴在一家小店做售后退款。每天她收到一张退款申请表，在一个后台里逐条处理，而后台里的每个动作都是真的：退出去的钱收不回来，发给买家的通知也收不回来。后台每天最多允许退 ¥1000，超出的会被拒绝。

给系统的东西很少：目的；一份七条的 rubric（结果要和她自己做的一样；每一单在动手之前都先查询；通知买家是最后一步且只通知一次；没有被拒绝的退款、没有退两次的订单；汇报的第一行给出当天的总数；周日给一份周汇总）；以及她的访问记录——大约六十轮 agent 对话、二十三条 shell 命令、二十三条聊天消息和五条浏览记录，其中约一半与任务无关。

没有给系统的，是写出这些记录时所依据的十七条标准。六条管当天之内的顺序与前置条件（先申请的先处理，而不是按表格的行顺序；买家自己原因的退货要扣运费；超过 ¥500 的升级；先退款再通知；退货没签收不退；不为了凑额度把一笔退款拆开）。五条管她希望别人怎么向她汇报（第一行就是结论；一单一行，不写过程；需要她处理的放最前面并标出来；金额写成 ¥ 加两位小数；不贴日志）。还有六条只在跨天时才存在（昨天没退成的单，今天按它最初的申请时间和新申请排成一队；主管批准的单，批准当天就退；寄回并签收之后的再次申请不算重复；已经退过款之后的再次申请才算；被驳回的要通知买家；周汇总遵守同样的汇报规矩）。

这个任务是特意设计成“看起来正确、但顺序不对的 workflow 会失败”的。图 4 的下半部分是周一。按申请时间处理，四单被退款，最晚申请的那一单等明天的额度。按表格的行顺序处理，每一个动作单看仍然是合法的，但一个较晚的申请先占掉了额度，一个较早的申请反而被挡在外面；后台最终停在另一个状态，到了周二，排在队首的也成了另一单。参考的一周和每一个“顺序类”探针，都是真的把后台跑一遍得到的：一次完全照她的规则，然后每条规则各一次、只忽略这一条。这样日志、状态和汇报三者始终互相一致——verifier 只有知道那条规则才抓得住探针，而不能靠发现文件之间对不上。

4 方法

采集：先授权。 每个来源在被发现时都不会被打开，只有本人允许之后才读取：本机和服务器上的 coding agent 日志、shell 历史、浏览器历史（只保留域名和路径）、导出的聊天记录、最近打开的文件、git 提交。屏



图 4：七天的样本。上：系统看得到的和被隐藏的。中：这一周；没退成的单、主管的审批、寄回的包裹把各天连在一起。下：同一个周一，按两种顺序处理。

幕录制和录屏视频导入是可选项。所有文本入库前都会脱敏。每个来源写的都是同一种事件格式。

编译：不调模型。 一天的事件被压成一条时间线和一批*主线之外的信号候选*。对话类信号只需要日志：agent 回复之后的一次纠正、一次要求给出证明、一次被中断的轮次、一次被拒绝的工具调用、一份用户自己写的任务规格。屏幕类信号需要录屏：看一眼另一个窗口却什么都没复制回来（这是核对；复制回来的则是取数）、连续撤销、人已经走开之后又回来重新保存的文件（这不需要任何标注，就给出了同一份产物的“被丢弃版”和“被保留版”）。检测器故意做成高召回的；每个命中只是一个留给 agent 去解读的指针，不是结论。

蒸馏：在别人的 agent 里做。 HT2WV 不调用模型 API。它写出一份任务书（产出、硬性约束、必须停下的情况、自检），然后在已安装的 coding agent 里开一个无头会话。一个会话写画像——这个人的工作习惯、验收风格、汇报风格、做事顺序，每条都带证据——并列出候选任务。然后每个任务一个会话，写出 skill。

这一对。 workflow 是一串阶段 prompt。verifier 有五层：*客观*（与人无关的正确性）、*顺序*（动作的先后与前置条件，在执行留下的日志上检查）、*风格*（这个人，跨任务）、*偏好*（这个人，这个任务）和*汇报*（汇报回来的东西的结构、长度与可读性）。每一条个人条目都必须引用记录，否则只能标成低置信度的假设。lint 这一步会拒绝这样的 verifier：它判这个人留下的产出不通过，或者它没有任何条目能判这个人丢弃的产出不通过。

执行与修订。 每个阶段作为一个独立的会话运行；verifier 的 prompt 附在所有会话后面，最后一个阶段负责跑可执行的检查。一次运行之后，这个人在 rubric 上的结论交给一个*迭代*会话，它可以修各个阶段，也可以补上有证据支撑的验收条目。在正式的优化循环里，修订 workflow 时 verifier 是按哈希冻结的，负责修订的会话看不到留出集上的结果；而 verifier 的任何改动，都要等本人批准之后才生效。

5 带隐藏风格的样本

一个样本是按“一个人存在的顺序”来构造的：先有风格，再有行为。我们先写人物、目的、rubric 和一份风格清单；然后写访问记录，让每一条风格留下它自然会留下的那种痕迹（一份被打回的草稿、一句“你核

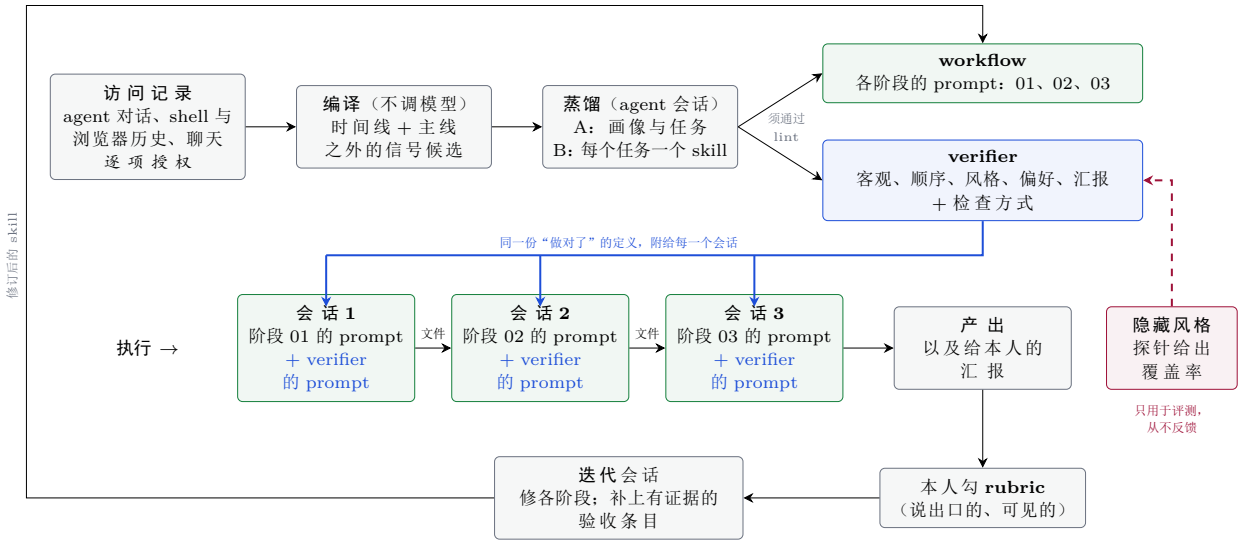


图 5: 流程。记录在授权之下采集，并在不调模型的情况下被压缩；agent 会话蒸馏出画像，并为每个任务蒸馏出 workflow 和 verifier；workflow 以多个会话的方式执行，每个会话拿到的都是同一份 verifier prompt。rubric 上的失败驱动修订；隐藏的风格条目只用于打分。

对过吗”、对同事说的一句话、一次被拒绝的动作），并混入与任务无关的日常活动。记录被渲染成各个来源的原生格式（agent 会话日志、带时间戳的 shell 历史、浏览器的历史数据库、聊天导出），再由真实的采集器读回来。风格清单和参考产出被隐藏。有效性检查要求：每一条风格至少影响了一条记录，并且参考产出能通过样本自己的 rubric。表 1 列出了六个样本。

样本	人物	任务	记录来源	隐藏条目	时间跨度
weekly-report	增长分析师	两份 csv → 周报	2 个 agent、shell、浏览、聊天	7	单步
swe-bugfix	后端工程师	issue → 修复、测试、提交信息	agent、shell、浏览、聊天	7	单步
guest-reply	民宿老板	客人消息 → 回复	agent、浏览、聊天	7	单步
video-publish	美食视频博主	拍摄笔记 → 发布信息	agent、浏览、聊天	7	单步
refund-ops	售后运营	一天的退款，带状态	agent、shell、浏览、聊天	11	1 天
refund-week	售后运营	七天的退款，带状态	agent、shell、浏览、聊天	17	7 天

表 1: 样本。在两个退款样本里，动作的顺序会改变最终状态。

6 初步结果

设置。 所有会话都是 Codex CLI 会话，模型为 `gpt-6-astra`，推理强度 `xhigh`，在一台服务器上运行。每个样本只跑了一次；单日的 *refund-ops* 没有跑，它的内容被那一周的周一覆盖。对单步样本，我们先执行建好的 skill，把 rubric 上的结论交给一个迭代会话，再执行一次；需要评审判断的 verifier 条目只在最后一轮评估。

单步任务太简单了。 四个 workflow 里有三个在第一次执行时就满足了整份 rubric；把评审条目算上之后，所有带探针的隐藏条目都被抓住了（表 2）。在这些样本里，产出是一个静态文件，做的顺序无关紧要，每条标准又几乎被本人在某次对话里原话说过。它们验证了管线是通的，除此之外说明不了多少——这也是我们去做退款样本的原因。

	rubric 完成度		风格覆盖率		参考产出
样本	第 0 轮	第 1 轮	只算检查脚本	加上评审条目	被接受
weekly-report	5/5	5/5†	6/6	6/6	是
swe-bugfix	4/5	5/5	1/6	6/6	**否**
guest-reply	5/5	4/5	4/6	6/6	**否**
video-publish	5/5	5/5	5/6	6/6	是

表 2：单步样本，各跑一次。每个样本有一条隐藏条目是过程类标准，没有针对产出的探针，不计分。† 最初被判为 2/5，原因是我们的一条 rubric 错误地要求列表符号；表中是修正 rubric 之后对同一份产出重新打的分。

只看覆盖率会被误导。 四个样本里有两个，经过一次修订之后的 verifier 拒绝了参考产出——也就是这个人本来会留下的那份回复或补丁。在 *guest-reply* 里，修订后的 verifier 在三条硬性条目上判参考回复不通过：提前入住的加收费用没有写明“视当天房态”这个条件；报出的总价没有包含宠物清洁费；产出里没有能证明“这条回复没有被发送”的执行记录。每一条单独看都说得通，但没有一条是她的标准。同一次修订还让 workflow 变得过于谨慎：它在一份正确的回复旁边又写了一份“已阻塞”的说明，丢掉了一条之前已经满足的 rubric。一个什么都判不通过的 verifier 能拿到满分的覆盖率；所以“参考产出被接受”必须和覆盖率一起报告，而修订这一步需要拿这个人留下的产出当回归集。

打分本身也很容易引入假象。 *weekly-report* 看上去从 5/5 掉到 2/5，其实是我们自己的问题：rubric 要求了这个人从没要求过的列表符号。我们把这件事留在表里，因为它和上一段是同一种失败，只不过发生在评测这一侧：一个过于具体的检查，冒充了一条标准。

那一周。 *refund-week* 共开了 47 个会话，历时 5 小时 22 分：1 个画像会话、1 个构建会话、19 个阶段会话、4 个修订会话和 22 个评审会话。建出来的 workflow 分三个阶段（对账、处理、交付），verifier 自有条目 10 条。**七天里填满了三天**，43 个按天的 rubric 判定里通过了 33 个（表 3）。没填满的四天有三种不同的原因，其中一种出在我们自己身上。

日	R1	R2	R3	R4	R5	R6	R7	填满	发生了什么
周一	✓	✓	✓	✓	✓	✓	–	是	
周二	✓	×	✓	×	✓	✓	–	否	给一单她不会动的旧单重发了通知
周三	✓	✓	✓	✓	✓	✓	–	是	经过一次修订之后
周四	×	×	✓	×	✓	×	–	否	第一个会话阻塞，什么都没做
周五	✓	×	✓	×	✓	✓	–	否	先清周四的积压，额度用尽
周六	✓	×	✓	×	✓	✓	–	否	把已在等额度的三单又升级了一遍
周日	✓	✓	✓	✓	✓	✓	✓	是	包括周汇总

表 3：那一周，一次运行。R1：后台自己生成的日志与状态、以及日报都在。R2：每一单的结果，都等于把她的规则用在这次运行自己留下的状态上所得到的结果。R3：做任何动作之前先查询过。R4：通知买家恰好一次、是最后一步、模板对得上；不该动的单不通知。R5：没有被拒绝的退款，也没有哪一单退了两次。R6：日报第一行与后台一致。R7（周日）：周汇总第一行与后台一致。没填满的那天之后，由一个修订会话看到当天的判定。周四的 R3、R5 之所以通过，只是因为什么都没做。

周二是 workflow 的错。周一被通知“先把货寄回”的一单仍在等签收，买家也没有再来申请。这样的单她是不动的；workflow 却把同一条通知又发了一遍。建出来的 verifier 在这一点上是对的：它检查“通知收尾”的那一条报出了“应为 0 次，实际 1 次”。但它是在最后一个会话里运行的，那时通知已经发出，而通知收不回来。针对不可逆动作的标准，放到交付时才检查是没有用的，它必须卡在动作之前。

周四有一部分是评测管线的错。第一个会话写了一份“已阻塞”说明，这一天什么都没做。它给出两条理由：workflow 要它读的一份引用文件不在运行目录里——这是我们的导出步骤漏拷的；另有一单的规则口

径不明，而这一单的两种读法其实会得到同一个结果。说明里写了其余五单不受影响，但管线只要看到“已阻塞”说明就结束这一天。如果文件在，这一天能不能填满，我们无法判断。

*周五和周六是周四的后果。*周五，这次运行在订单表里看到了周四那五条没处理的申请，按最初申请时间把它们排在最前面先办，这正是她自己对待遗留工作的规则。当天的额度因此用尽，周五自己的三单被升级而没有退成。在我们的标准答案里，每天的申请只投递一次；它不知道周四的申请还悬着，于是把十单里的八单判成了错。如果把周四的申请和审批仍留在收件箱里重新评判，十单里只有一单不同：主管的一条驳回只在周四投递过，这次运行从头到尾没有见过它。周六，这次运行把已经在等额度的三单又升级、又通知了一遍。这和周二是同一类错误，只是落在周二那次修订没有覆盖到的分支上。

*周日填满了，*包括周汇总；但这一周收尾时是亏的：退款 20 单合计 ¥5556.00、5 单待处理，而参考的一周是 23 单、¥6026.00、1 单待处理。有一位买家始终没有被告知申请已被驳回；一共发出了 41 条通知，而她会发 36 条。

隐藏的风格与参考产出。 一周结束时的 verifier 抓住了 17 条隐藏风格里的 16 条（只算检查脚本是 12 条）：当天顺序 6/6，汇报 4/5，跨天规则 6/6。它同时*拒绝了我们探测的全部五天参考产出*。我们把每一条不通过的条目都追到了来源。

- *出在我们这边。*参考日报的第一行写的是“处理了几单”，而她留下的日报和隐藏条目 S6 写的都是“几条申请”。两条脚本条目和一条评审条目据此判它不通过，判得对。把参考产出改正之后，用同一个 verifier 的脚本条目重跑（不涉及模型）：五天里有三天全部通过，脚本覆盖率从 12/17 升到 17/17。运行中漏掉的那一条（金额格式）其实是被遮住了：能发现它的那个条目，在参考产出上本来就不通过。
- *出在修订。*另外两天脚本仍然不通过，理由是参考产出没有去查询、也没有在日报里列出那些她根本不动的单。这条要求是修复周二错误的那次修订加进去的。
- *没有人要求过的评审条目。*有两条硬性的评审条目在每一天都不通过。一条从第一次构建起就在，要求日报在没有独立回执时声明“后台来源未证实”；另一条是某次修订加的，要求留下“先对账、再写日报”的证据。周日还多出第三条，它继承自画像，因为周汇总给前几天的数字标了日期而判其不通过。她的记录里没有这些标准。（评审条目没有在改正后的参考产出上重跑；它们当时给出的理由都与第一行无关。）

verifier 为什么只变得更严。 有两个机制。lint 这一步会拿她留下的产出去回放脚本条目，这一周里它们一直是通过的；但评审条目从不在她留下的产出上运行，所以一条硬性的评审条目可以和这些产出相矛盾而不被发现。另外，修订用的 prompt 禁止放宽或删除条目（除非它与 rubric 直接矛盾），于是 verifier 从 10 条长到了 16 条，中间没有任何一次被拿去对照“她会接受的东西”。代价不只落在 verifier 上。到周日，workflow 写的周汇总第一行先声明自己的合计“未证实”，表里还多了一列“证据范围”。它通过了那条只核数字的 rubric，读起来却不像她写的。这是图 1 里的三角在倒着转：一条本人从没批准过的标准，把 workflow 从她身边推开了。

这次运行之后我们改了什么。 导出步骤现在会带上 skill 的引用文件，lint 会标出执行会话打不开的链接。参考日报的第一行改成了她留下的日报的写法。哪一天什么都没做，这一天的申请和审批会留在第二天的收件箱里。这一周还没有在这些改动之上重跑；表 3 是当时那次运行的原样。

需要说明的限制。 这些数字来自一次运行、一个模型，以及由我们自己为了体现风格而写出来的记录，所以它们更接近上界，而不是估计值。目前还没有真人参与。系统还没有在真人的记录上、带着 agent 后端完整地跑过一遍；在作者自己的机器上，只有确定性的那几个阶段在真实日志上运行过。

7 相关工作

*从演示到 skill。*现在的产品能把一次录下来的、有时带讲解的演示变成可复用的 skill [1, 2, 3]。它们是一次性的，需要人专门演示一遍，记录的是主线。*从被动轨迹到任务模型与用户模型。*任务模型归纳 [4] 能把未标注轨迹里交错的任务拆开，并构建带控制流的目标层级；通用用户模型 [5] 从截图里维护关于用户的、带

置信度的命题。两者都不产出验收标准。一项从 GUI 轨迹里挖掘 skill 文件的研究报告说，聚出来的簇可读，但对下游几乎没有帮助 [6]；我们把它读作“缺一个目标函数”。从 *agent 轨迹里得到 workflow*[7]，其“成功”由基准的评测器给出。*带 verifier 的 skill*。EvoSkills[8] 让 skill 生成器与一个替代 verifier 共同演化，并报告去掉 verifier 后损失很大；agentic rubrics[9] 让 agent 读完仓库后写出一份带权重的检查清单。它们的 verifier 是从任务推出来的，不是从某个人那里来的。*从隐式反馈里学偏好*。PRELUDE[10] 从用户对 agent 产出的编辑里推断潜在偏好，并用来约束生成。我们把信号从“编辑”扩展到纠正、拒绝和做事顺序上的习惯，并把结果用作一个可执行的验收测试。[本节依据的是摘要和二手综述；投稿前需要逐条对照原文核实。]

8 局限与下一步

多样化的 working group 和本地蒸馏目前都还不存在：没有任何开源模型在这些样本上被训练或评测过，本文报告的所有会话都运行在托管模型上。核心主张——在别人的 verifier 之下优化出来的 workflow，在本人这里被接受得更少——还没有被检验；这需要两个人（或者同一任务上的两种风格）和一次交叉评测。合成的记录比真实的记录友好：大多数标准都被本人直接说了出来，也很少有东西与之矛盾。更难的样本应当包含：只被做出来、从没被说出来的标准；随时间变化的标准；以及习惯不同、却出现在同一个聊天里的同事（退款样本里已经有几处）。对留出材料的隔离靠的是工作目录，而不是操作系统级的沙盒。最后，verifier 最多只能和这个人自己的盲点一样好；客观层的存在，以及每一条条目都对本人可见，是目前对这一点的回答。

那一周的运行指向方法本身的三处改动，目前都还没有做。评审条目在被允许设为硬性之前，应当像脚本条目那样先在本人留下的产出上回放一遍。修订必须能够删除或放宽条目，并以本人留下的产出作为回归集，否则 verifier 只会越收越紧。针对不可逆动作的条目，必须在动作之前检查，而不是在交付时检查。

参考文献

[1] Anthropic. Record a Skill in Claude Cowork. Product feature, 2026.

[2] OpenAI. Record & Replay in the Codex desktop app. Product feature, 2026.

[3] Microsoft. Skill Recorder. https://github.com/microsoft/skill-recorder, 2026.

[4] Y. Jiang, Z. Z. Wang, R. Chen, D. Yang. Inducing Task Models from Computer-Use Traces. arXiv:2608.20319, 2026.

[5] O. Shaikh et al. Creating General User Models from Computer Use. arXiv:2505.10831, 2025.

[6] Mining skill files from GUI trajectories. arXiv:2606.20363, 2026. [作者、准确标题]

[7] Z. Z. Wang et al. Agent Workflow Memory. arXiv:2409.07429, 2024.

[8] EvoSkills: co-evolving skills and surrogate verifiers. arXiv:2604.01687, 2026. [作者、准确标题]

[9] Agentic Rubrics as Contextual Verifiers for SWE Agents. arXiv:2601.04171, 2026. [作者]

[10] G. Gao, A. Taymanov, E. Salinas, P. Mineiro, D. Misra. Aligning LLM Agents by Learning Latent Preference from User Edits. NeurIPS, 2024.